two nodes extracted at step (B) as its children. By convention, we assume that the smallest node will be added as left child and the second-smallest as right child (but any consistent convention works here). The newly-created node will hold the union of the set of characters in its children as value, and the sum of the two priorities as priority.
(D) Finally, we can add the new root for this subtree back to the priority queue.
Note that the nodes in the heap are showed in sorted order, but just for the sake of simplicity: as we stated, the order in which nodes are stored inside a priority queue is an implementation detail, the contract for PQ's API only guarantees that when we dequeue two elements, those will be the ones with the smallest frequencies.

Back to our algorithm, you can see how the tree in figure 2.15 is constructed bottom-up, and lines #2 to #4 in listing 2.15 take care of the first step, creating the leaves of the tree and adding them to the priority queue.

Now, from line #5 we enter the core of the algorithm: until there is only one element left in the queue, we extract the top `TreeNode` entries, in lines #6 and #7: as you can see in figure 2.16.B, those two elements will be the subtrees with the lowest frequencies so far.

Let's call this subtrees `L` and `R` (the reason for these names will be apparent soon).

Figure 2.16.C shows the actions performed in lines #8 to #10 of our pseudocode: a new `TreeNode` is created (let's call it `P`)